

Name : P.Jeeva

Reg No:192412636

Course Code:ITA0609

CO5-AT2

Data Cleaning and Transformation on Time-Series Data

Aim

To perform data cleaning and transformation on time-series data by handling missing timestamps, interpolating missing values, removing outliers, and resampling the data at fixed intervals using Pandas.

Objective

- To convert timestamps into Date-Time format.
- To set time as the DataFrame index.
- To maintain chronological order.
- To identify missing timestamps.
- To interpolate missing values.
- To detect and remove outliers.
- To resample the data at fixed intervals.

Algorithm / Pseudocode

START

1. Load the time-series dataset.
2. Convert the timestamp column into Date-Time format.

3. Set the timestamp column as the DataFrame index.
4. Sort the data according to the timestamp.
5. Remove duplicate timestamps and invalid values.
6. Create a regular time interval.
7. Reindex the data using the regular time interval.
8. Identify missing values caused by missing timestamps.
9. Interpolate the missing values.
10. Calculate Q1 and Q3 values for detecting outliers.
11. Calculate $IQR = Q3 - Q1$.
12. Identify values outside the range:
 $Lower\ Limit = Q1 - 1.5 \times IQR$
 $Upper\ Limit = Q3 + 1.5 \times IQR$
13. Remove the detected outliers.
14. Resample the cleaned data at the required fixed interval.

15. Sort the final data chronologically.

16. Check for remaining missing or invalid values.

17. Display the cleaned time-series data.

END

Python Code

```
import pandas as pd
```

```
import numpy as np
```

```
# Create time-series data
```

```
data = {
```

```
    'Timestamp': [
```

```
        '2026-01-01 10:00',
```

```
        '2026-01-01 10:05',
```

```
        '2026-01-01 10:15',
```

```
        '2026-01-01 10:20',
```

```
        '2026-01-01 10:25'
```

```
    ],
```

```
    'Temperature': [25, 26, 30, 31, 100]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
# Convert timestamp to datetime
df['Timestamp'] = pd.to_datetime(df['Timestamp'])

# Set timestamp as index
df = df.set_index('Timestamp')

# Sort chronologically
df = df.sort_index()

# Create regular 5-minute intervals
df = df.resample('5min').asfreq()

# Interpolate missing values
df['Temperature'] = df['Temperature'].interpolate()

# Detect outliers using IQR
Q1 = df['Temperature'].quantile(0.25)
Q3 = df['Temperature'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# Remove outliers
df.loc[
    (df['Temperature'] < lower) |
```

```

(df['Temperature'] > upper),
'Temperature'
] = np.nan

# Interpolate after removing outliers
df['Temperature'] = df['Temperature'].interpolate()

print("Cleaned Time-Series Data:")
print(df)

```

Explanation of Important Functions

Function	Purpose
pd.to_datetime()	Converts values into Date-Time format
set_index()	Sets timestamp as index
sort_index()	Arranges data chronologically
resample()	Creates fixed time intervals
interpolate()	Fills missing values
quantile()	Calculates Q1 and Q3
loc[]	Filters and modifies data

Expected Output

The output will contain timestamps at regular **5-minute intervals**. The missing timestamp at **10:10** will be created and its temperature will be interpolated. The extreme temperature value **100** will be detected as an outlier and replaced through interpolation.

... Cleaned Time-Series Data:		
Timestamp		Temperature
2026-01-01	10:00:00	25.0
2026-01-01	10:05:00	26.0
2026-01-01	10:10:00	28.0
2026-01-01	10:15:00	30.0
2026-01-01	10:20:00	31.0
2026-01-01	10:25:00	31.0

Result

Thus, the time-series data was successfully cleaned and transformed by handling missing timestamps, interpolating missing values, removing outliers, and maintaining chronological consistency.

2. FEATURE SELECTION ON A DATASET

Aim

To implement feature selection on a dataset by calculating correlation between features, removing redundant or irrelevant attributes, normalizing the data, and selecting significant features to reduce dimensionality and improve model performance.

Objective

1. To load a dataset using Pandas.
2. To separate input features and target variable.
3. To normalize the feature values.
4. To calculate correlation between features.
5. To identify highly correlated features.
6. To remove redundant features using a threshold.
7. To select the important features.
8. To validate the selected features using a machine learning model.
9. To compare model performance before and after feature selection.

Dataset Used

The dataset contains information about customers.

Age Salary Experience StudyHours Purchased

20	20000	1	2	0
25	25000	2	3	0
30	30000	3	4	0
35	35000	4	5	1
40	40000	5	6	1
45	45000	6	7	1
50	50000	7	8	1
55	55000	8	9	1

Target variable: Purchased

Input features: Age, Salary, Experience, StudyHours

Pseudocode / Algorithm

START

1. Load the dataset.
2. Separate the input features from the target variable.
3. Check the dataset for missing or invalid values.
4. Normalize the input features using Min-Max normalization.

5. Calculate the correlation matrix between all features.
6. Set a correlation threshold of 0.80.
7. Compare the correlation values between feature pairs.
8. If the correlation between two features is greater than 0.80, identify one feature as redundant.
9. Remove the redundant features.
10. Store the remaining significant features.
11. Split the dataset into training and testing data.
12. Train a Logistic Regression model using all features.
13. Calculate the accuracy of the model.
14. Train another Logistic Regression model using only the selected features.
15. Calculate the accuracy of the second model.
16. Compare both accuracies.

17. If the selected-feature model gives similar or improved performance, accept the selected features.

18. Display the final selected features and accuracy.

END

Normalization

Normalization is used to bring different features into a common scale.

Min-Max Normalization Formula

$$\text{Normalized Value} = (X - X_{\min}) / (X_{\max} - X_{\min})$$

The normalized values generally fall between **0 and 1**.

For example, if:

- $X = 30$
- $X_{\min} = 20$
- $X_{\max} = 55$

Then:

$$\text{Normalized Value} = (30 - 20) / (55 - 20)$$

$$= 10 / 35$$

$$= 0.286$$

Correlation

Correlation shows how strongly two features are related.

The correlation value ranges from **-1 to +1**.

- **+1** → Strong positive relationship

- **0** → No linear relationship
- **-1** → Strong negative relationship

In this program, the correlation threshold is:

Threshold = 0.80

If two features have correlation greater than **0.80**, one of them can be removed because they provide similar information.

Drag points

2468102468XY

Linear correlation is positive ($r = 0.98$).

NegativeNear zeroPositive

NegativeNear zeroPositive

Give feedback

Python Program

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score
```

```
# Small dataset
```

```
data = {
    'Age': [20, 25, 30, 35, 40, 45, 50, 55],
    'Salary': [20000, 25000, 30000, 35000, 40000, 45000, 50000, 55000],
    'Experience': [1, 2, 3, 4, 5, 6, 7, 8],
    'StudyHours': [2, 3, 4, 5, 6, 7, 8, 9],
```

```
'Purchased': [0, 0, 0, 1, 1, 1, 1, 1]  
}
```

```
df = pd.DataFrame(data)
```

```
print("Dataset:")
```

```
print(df)
```

```
# Separate features and target
```

```
X = df.drop('Purchased', axis=1)
```

```
y = df['Purchased']
```

```
# Normalize
```

```
scaler = MinMaxScaler()
```

```
X_scaled = pd.DataFrame(  
    scaler.fit_transform(X),  
    columns=X.columns  
)
```

```
# Calculate correlation
```

```
correlation = X_scaled.corr()
```

```
print("\nCorrelation Matrix:")
```

```
print(correlation)
```

```
# Set correlation threshold
```

```
threshold = 0.80
```

```
upper = correlation.where(  
    np.triu(np.ones(correlation.shape), k=1).astype(bool)  
)
```

```
# Find redundant features
```

```
remove = [  
    column for column in upper.columns  
    if any(upper[column].abs() > threshold)  
]
```

```
# Select features
```

```
X_selected = X_scaled.drop(columns=remove)
```

```
print("\nRemoved Features:")
```

```
print(remove)
```

```
print("\nSelected Features:")
```

```
print(X_selected.columns.tolist())
```

```
# Split data
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled, y, test_size=0.25, random_state=1  
)
```

```
Xs_train, Xs_test, ys_train, ys_test = train_test_split(  
    X_selected, y, test_size=0.25, random_state=1  
)
```

```
# Model using all features
```

```
model1 = LogisticRegression()
```

```
model1.fit(X_train, y_train)
```

```
accuracy1 = accuracy_score(  
    y_test,  
    model1.predict(X_test)
```

```
)
```

```
# Model using selected features
```

```
model2 = LogisticRegression()
```

```
model2.fit(Xs_train, ys_train)
```

```
accuracy2 = accuracy_score(  
    ys_test,  
    model2.predict(Xs_test)
```

```
)
```

```
print("\nAccuracy using all features:", accuracy1)
```

```
print("Accuracy using selected features:", accuracy2)
```

```
print("\nFeature Selection Completed Successfully!")
```

Explanation of Functions Used

Function	Purpose
<code>pd.DataFrame()</code>	Creates the dataset
<code>drop()</code>	Separates/removes columns
<code>MinMaxScaler()</code>	Normalizes feature values
<code>fit_transform()</code>	Learns and transforms the data
<code>corr()</code>	Calculates correlation
<code>np.triu()</code>	Selects the upper correlation matrix
<code>train_test_split()</code>	Splits data into training and testing sets
<code>LogisticRegression()</code>	Creates the classification model
<code>accuracy_score()</code>	Calculates model accuracy

Expected Output

The program displays:

Dataset:

Correlation Matrix:

Removed Features:

[...]

Selected Features:

[...]

Accuracy using all features: ...

Accuracy using selected features: ...

Feature Selection Completed Successfully!

The exact accuracy values may vary depending on the train-test split.

```
... Dataset:
   Age  Salary  Experience  StudyHours  Purchased
0   20   20000           1           2           0
1   25   25000           2           3           0
2   30   30000           3           4           0
3   35   35000           4           5           1
4   40   40000           5           6           1
5   45   45000           6           7           1
6   50   50000           7           8           1
7   55   55000           8           9           1

Correlation Matrix:
           Age  Salary  Experience  StudyHours
Age         1.0    1.0    1.0    1.0
Salary      1.0    1.0    1.0    1.0
Experience  1.0    1.0    1.0    1.0
StudyHours  1.0    1.0    1.0    1.0

Removed Features:
['Salary', 'Experience', 'StudyHours']

Selected Features:
['Age']

Accuracy using all features: 0.5
Accuracy using selected features: 0.5

Feature Selection Completed Successfully!
```

Validation

The model is trained twice:

Model 1

Uses **all available features**.

Model 2

Uses only the **selected features**.

The two accuracy values are compared.

If the selected-feature model gives similar or better accuracy while using fewer features, the feature-selection process is considered successful.

Advantages

1. Reduces the number of features.
2. Removes redundant information.
3. Reduces model complexity.
4. Can improve model performance.
5. Makes the dataset easier to analyze.
6. Reduces computational requirements.

Result

Thus, feature selection was successfully implemented by normalizing the dataset, calculating feature correlation, removing redundant features using a threshold, selecting significant features, and validating the selected features using Logistic Regression accuracy.